Automated test generation using symbolic execution

Contents

- 10.1. Learning outcomes of this chapter
- 10.2. Chapter introduction
- 10.3. Symbolic execution
- 10.4. Dynamic symbolic execution
- 10.5. Into the future...

10.1. Learning outcomes of this chapter

At the end of this chapter, you should be able to:

- Explain the concepts of symbolic execution and dynamic symbolic execution, and discuss their strengths and weaknesses.
- Compare and contrast (dynamic) symbolic execution with other forms of testing, such as random testing and equivalence partitioning.
- Manually perform symbolic execution on a small program, deriving the path condition for that program.
- Compare and contrast symbolic execution and dynamic symbolic execution.

10.2. Chapter introduction

One of the downfalls of standard testing is that, even if we run 1 million tests and have no failures, there is no guarantee that the next test we run will not fail. Recall Dijkstra's famous quote from Chapter 1:

> *"Program testing can be used to show the presence of bugs, but never > to show their absence!"* — Edsger W. Dijkstra[1]

Footnotes

[1]: In Notes on Struc Programming.

However, what if we *could* run every possible input on a program, even if there were an infinite number?

Symbolic execution is one approach to software verification that aims to do just that. The idea behind symbolic execution is that, instead of executing a program with concrete values, we execute it with *symbolic* values, which effectively execute multiple values at one time.

In this chapter, we will look at symbolic execution, and its variant *dynamic* symbolic execution, and how they can be used to verify software.

10.3. Symbolic execution

10.3.1. Early symbolic execution

Although first proposed by James King in 1976[2], symbolic execution is still in its infancy, and has only recently emerged as a realistic way to verify software.

Footnotes

[2]: J. C. King, "Syml execution and progra Communications of t 19(7):(385–394), 19

While early work by researchers such as King were promising, symbolic execution suffered from two major problems:

1. Computers were small with limited memory and slow processors, compared to today's machines.
2. The constraint solving tools required for symbolic execution were limited.

Modern computers are significantly faster than those in the 70s, and memory is cheap. Further, constraint solving

Skip to main content

become a hot topic of research again.

10.3.2. Constraint solving

A key technology required for symbolic execution is *constraint solving*. Constraint solving is an approach to generating solutions for mathematical problems expressed as constraints over a set of objects.

Each constraint to be solved has three parts: (1) a set of variables; (2) a *domain* for each variable (e.g. the integers, characters, or real numbers); and (3) the set of the constraints.

The easiest way to understand constraint solving problems is via an example. Consider the following constraint over the set of integers and variables x and y:

$$x \leq 0,\ y > 0,\ x + y > 100 \qquad (10.1)$$

Constraint solvers can be used to answer three types of questions:

1. Is there a solution to this problem?
2. If "yes" to 1, what is a solution to the problem?
3. Is one constraint entailed by another?

There are, of course, many solutions to the constraint in Equation 10.1 so a constraint solver will answer "yes" to the first question. If asked to provide a solution to the constraint, it could return a number of answers, such as $x = 0,\ y = 101$, or $x = -10000,\ y = 200000$ (although the former more likely in most constraint systems!).

An example of a constraint that is not solvable is:

$$x < 0,\ y < 0,\ x + y > 0 \qquad (10.2)$$

If x and y are both negative, then their sum cannot be positive. As such, a constraint solver will answer that this is *unsatisfiable*.

For checking constraint entailment, we ask the solver if constraint C *entails* constraint D, written $C \vdash D$. Entailment is similar to logical implication, and $C \vdash D$ is asking: is any solution for C also a valid solution for constraint D. In essence, if C stronger than D? That is, if everything on the left of $\vdash$ is true, does that mean that everything on the right is also true?

As an example, the entailment $x > 0 \vdash x \geq 0$ holds: if x is strictly positive, then it is also weakly positive. That is, any value of x that satisfies $x > 0$ also satisfied $x \geq 0$.

However, the inverse, $x \geq 0 \vdash x > 0$, does not hold, because $x = 0$ is a solution for the former but not the latter.

Constraint solvers can also answer more complicated problems, such as:

$$x < 200, y > 200, z = x - 100 \quad \vdash \quad y \geq 2z$$

A constraint solver should infer that $z < 100$, and therefore $2z < 200$; so because $y > 200$, it must always by the case that $y \geq 2z$.

10.3.3. Symbolic states

Constraints, such as those used in Equation 10.1 can be used to assign *symbolic* values to program variables. By symbolic, we mean values that represent possible more than one value. For example, the constraint $x < 0$ can be a symbolic value for a variable x, indicating that x is negative. This is in contrast to a *concrete* value of x, such as $x = 1$, which defines a specific value for x.

When we perform standard software testing, we provide a program with concrete inputs, and those inputs are executed. Exploring the entire input space with concrete values is impossible for even the most trivial examples.

At any point during execution of a program, the program will have a *state*, which is a mapping from all variables to their concrete values at that time. As a program executes, calculations (instructions) are executed on the variables in that state, and their values are updated. For example, the concrete state may be $x = 5, y = 10$, and the instruction $x := y + 1$ (assignment) will update the state to $x = 11, y = 10$.

The key idea behind symbolic execution is to replace the concrete state with a *symbolic state*, in which the variables map to symbolic values, and to perform calculations on those symbolic values. By doing this, a program can be executed for many

Skip to main content

10.3.4. Some examples

Example 70

As an example, consider the example program in Figure 10.1, which swaps the values of two integers, `x` and `y`, if and only if `x` is less than `y`. The result is that `x` is always greater than `y`.

```
1.  void swap(int x, int y)
2.  {
3.    if (x < y) {
4.      x = x + y;
5.      y = x - y;
6.      x = x - y;
7.    }
8.    assert(x >= y);
9.  }
```

Figure 10.1: A program for swapping two integers.

We can execute this program on a concrete state, providing concrete values for `x` and `y`; for example, `x = 4` and `y = 10`. The concrete state of the program would be updated as follows:

Line	Program fragment	Variable values		Notes
		x	y	
1.	`void swap(int x, int y)`	4	10	
3.	`if (x < y)`	4	10	(branch true)
4.	`x = x + y`	14	10	
5.	`y = x - y`	14	4	
6.	`x = x - y`	10	4	
8.	`assert(x >= y)`	10	4	(assertion true)

Executing the program for this pair of inputs will execute one path out of two, but for just one possible execution of that path. Any input in which `x < y` will also execute that path. If we are testing, we would (hopefully) select a value such that `x >= y`, covering the other path. In both cases, the assertion at the end of the function will be true.

However, if we replace the concrete state with a symbolic state, starting with the variable `x` and `y` being completely unconstrained, we can cover every possible input by symbolically executing just the two paths. At the branch, we split the symbolic execution into the two paths, executing all inputs on both paths by executing all instructions symbolically. At each point in time, the value of the variables is represented as a constraint, called the *path constraint*, which summarises all possible values of the program variables at that point.

The following illustrates how the symbolic state is modified during execution. In this illustration, we use capitalised variable names, `X` and `Y`, to represent the symbolic values of `x` and `y` at the start of the execution:

Line	Program fragment	Path constraint		Other
		Path `x < y`	Path `x >= y`	
1.	`void swap(int x, int y)`	`x=X, y=Y`		(Only one path so far)
3.	`if (x < y)`	`x=X, y=Y, X<Y`	`x=X, y=Y, X>=Y`	(Fork into two paths)
4.	`x = x + y`	`x=X+Y, y=Y, X<Y`	–	
5.	`y = x - y`	`x=X+Y, y=X, X<Y`	–	
6.	`x = x - y`	`x=Y, y=X, X<Y`	–	
8.	`assert(x >= y)`	`x=Y, y=X, X<Y`	`x=X, y=Y, X>=Y`	(assertion true)

At line 5, the assignment `y = x - y` means that `y=(X+Y)-Y`, which simplifies to just `y=X`. A similar simplification occurs at line 6. The constraint solvers are used to simplify these expressions.

From the symbolic execution above, we can see that the assertion `x >= y` holds at the end of both parts. Because symbolic values are used instead of concrete values, both of the paths are executed for all possible values of `x` and `y`. Therefore, we have executed the program for all possible inputs, and shown that the assertion at line 8 will always be true; that is, there is no test input that will violate it. We can show this by taking the path constraints at the assert statement and checking if they entail the assertion. The entailments `x=Y, y=X, X<Y` $\vdash$ `x >= y` and `x=X, y=Y, X>=Y` $\vdash$ `x >= y` both hold, so the assertion always holds.

If the programmer had made a mistake on line 6 and used addition instead of subtraction, the final constraint for the first path would be: `x = (X+Y) + X, y = X, X < Y`, and the assertion would be violated. The constraint solver can easily find a solution for the pair of symbolic variables (X, Y), say X = -5 and Y =4, that satisfies the path constraint but it violates the assertion. Indeed, with this solution: x - y = (2X + Y) - X = X + Y = -5 + 4 = -1. It means x - y < 0, violating x >= y.

Skip to main content

Example 71

As a more complicated example, consider the following function [3] written in C, which allocates memory to a pointer:

```
 1. int* func(int x; int y)
 2. {
 3.   int* p = 0;
 4.   int s = x + y;
 5.   if (s != 0) {
 6.     p = malloc(s);
 7.   }
 8.   else if (y == 0) {
 9.     p = malloc(x);
10.   }
11.   return p;
12. }
```

A common property for symbolic execution tools are used to check is whether functions can return null. Let's assume then that we want to analyse whether this function can return `null` (that is, the value of `p` is 0).

Figure 10.2 shows the symbolic execution tree for this program. There are three possible paths through the program. The constraints `x=X` and `y=Y` are in every node, and have been omitted for readability.

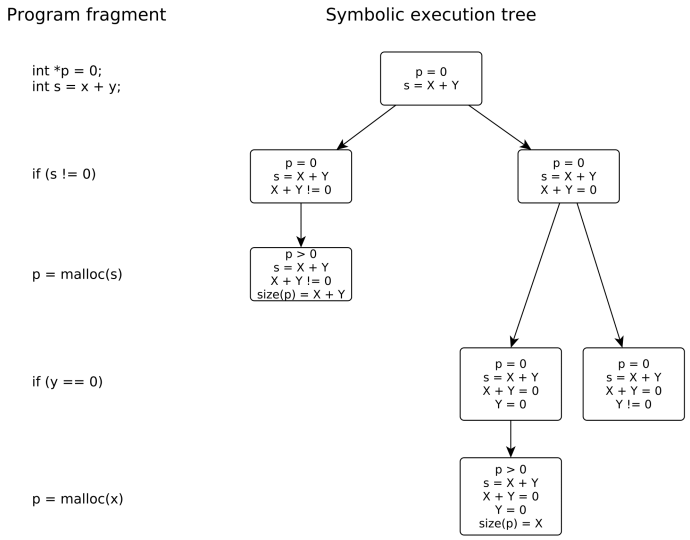


Figure 10.2: Execution tree for the function func

To check whether the pointer can possibly be null, we have to check whether for at least one path, whether the final path constraint conjoined with the constraint `p=0` is satisfiable. If it is satisfiable, there is at least one execution of the program that results in a null pointer.

From the execution tree, it is clear that the path on the right is one such path, with `p=0` being part of the path constraint.

10.3.4.1. A real fault

Symbolic execution can find considerably more subtle faults than the one above. For example, consider the following program fragment from the GNU CoreUtils package.

```
1322:     s = xmalloc(MAX(8, chars_per_input_tab));
....
2655:     width = chars_per_c - (input_position % chars_per_c);
2666:
2667:     if (untabify_input)
2668:     {
2669:       for (i = width; i; --i)
2670:         *s++ = ' ';
2671:       chars = width;
2672:     }
```

In this case, the program may be asked to "untabify" some input, meaning to remove tabs and replace them with spaces.

Footnotes

[3]: Adapted from th
http://martinsprogra
gspot.com.au/2011/1
execution.html

Skip to main content

incorrect assumption that `width` is always between 0 and `chars_per_c - 1`. When `input_position` is positive, this assumption holds; however, because backspaces are permitted in the input string, `input_position` can be negative, meaning that `width` can become larger than the size of `s`, and an overflow can occur at line 2670, as spaces are added past the end of the string.

This subtle fault was found by the KLEE symbolic execution tool[4] in 2008, and had been in the GNU CoreUtils package at least since it was first uploaded onto a CVS repository in 1992, despite the CoreUtils package having an unusually comprehensive test suite, and having been used for many years.

Such subtle faults are difficult to detect using dynamic software testing, and are where symbolic execution can be useful.

Footnotes

[4]: See the following details on KLEE: C. C Dunbar, and D. Engle Unassisted and Auto Generation of High-(Tests for Complex Sy Programs." OSDI. Vo

10.3.5. Symbolic test oracles

Symbolic execution is only a way to explore test inputs for a program, but it cannot tell us whether that program meets its requirements. For this, we need the equivalent of a test oracle for symbolic execution. In the example above, we use assertions as test oracle; however, most programs do not contain useful assertions that can be used.

Standard test oracle solutions, discussed in Chapter 6, cannot necessarily be applied directly, because they are designed for concrete examples, not symbolic examples.

Despite this, some of these solutions would work well. For example, if we have a golden program, then we can execute a path the using symbolic execution, and compare the final path constraint to the path constraint generated when executing the same symbolic input on the golden program. Also, the idea of metamorphic oracles can be applied to symbolic execution: for some programs, a specific symbolic value may return a particular output.

In most cases, symbolic execution is used to look for *generic* faults, such as accessing arrays out of bounds, dividing by zero, returning null pointers, or certain security violations. These can be checked for any program using a generic "oracle", and empirical evaluation shows that they work very effectively. Cadar et al.[5] found many faults in the GNU CoreUtils package, such as the one discussed above, several of which had been present for over a decade.

Footnotes

[5]: See the following details on KLEE: C. C Dunbar, and D. Engle Unassisted and Auto Generation of High-(Tests for Complex Sy Programs." OSDI. Vo

10.3.6. Test input generation

Recall from earlier in this section that constraint solvers can be asked to provide a solution for a constraint. As such, an important part of symbolic execution is test input generation: test inputs can be generated by asking the constraint solver for a solution to the path constraint.

For example, consider the earlier program that allocated memory to the pointer `p`. We can ask for test inputs for each path, giving us tests that achieve path coverage for the program.

More importantly, if our symbolic execution tool finds that the program can indeed return a null pointer, then we can also ask for a *concrete* test input that produces this case. This provides programmers with a value concrete case for debugging, which are much easier for humans to reason about than symbolic traces.

In the example, the fourth path results in a null point (`p = 0`), and we can ask the constraint solver to generate inputs for `x` and `y` can be solving the path constraint in the final node of that path:

$$x = X, y = Y, p = 0, s = X + Y, X + Y = 0, Y! = 0$$

Thus, the constraint solver must generate values `X` and `Y` equal 0, but `Y` itself does not. Any values such that `X = -Y` will suffice, such as `X = 1` and `Y = -1`. Execute these inputs on the program will return a null pointer.

10.3.7. Limitations

There are several challenges related to the scalability of symbolic execution, limiting their uptake, but which also present some interesting research problems.

Constraint solving: Symbolically executing all inputs on a path is expensive due to the cost of the underlying constraint solving. Current evaluations put the cost of symbolically executing a path at about 80 times that of concrete execution. This means that even a small set of unit tests covering a set of paths that take one minute to execute would take over an hour to symbolically execute.

There are two general solutions that significantly reduce the execution cost of symbolic execution:

1. *Removing unnecessary constraints*: Each branch in a program generally depends on a small number of variables (1 or

Skip to main content

that are not relevant.

For example, consider a symbolic execution tool encountering a branch `if (x < 0)`, with the path constraint $x + y > 0, z < 2a, y < 10$, and wants to check if it is feasible that `x` can be less than 0, the constraint $z < 2a$ can be removed from this check, as it will not influence the result.

2. *Caching solutions*: Branches in programs tend to have lots of constraints that are similar to each other. As such, when solutions are required; e.g. generating a test input; the symbolic execution tool can cache solutions are try to reuse them later.

   The field of constraint solving is a very active research area, and much progress is being made to speed up how efficiently constraints can be solved; however, the problem will persist, as reasoning over an entire collection of inputs will always take much longer than a single input.

Path explosion: The examples that we have seen so far are toy examples used to illustrate how symbolic execution works. If we want our symbolic execution tool to explore all possible paths through a program, the resulting number of paths explodes exponentially as the number of branches in the program increases.

For example, consider the control-flow graph in Figure 10.3 from a real program that is non-trivial, but not unusually large. Exhaustively symbolically executing every path through this program is not feasible.

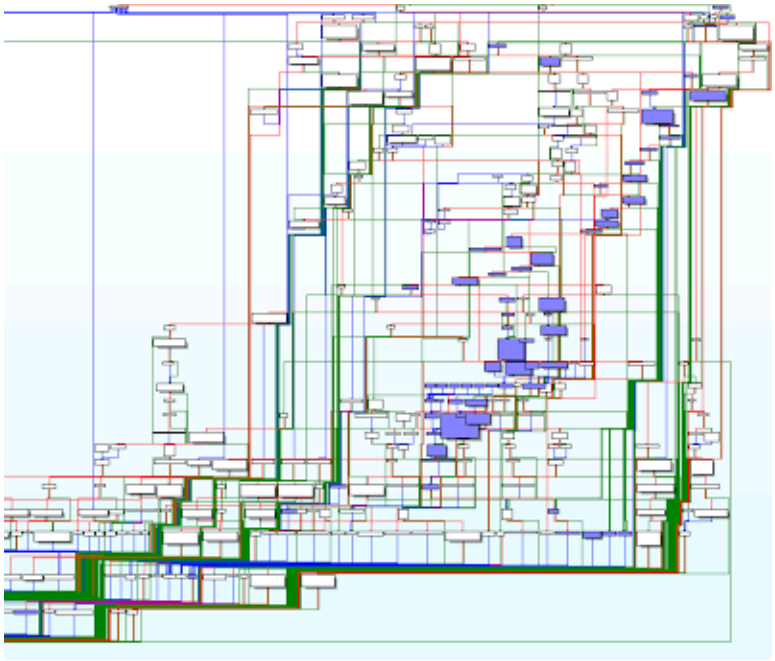


Figure 10.3: A control-flow graph for a non-trivial program.

Much research has gone into mitigating the path explosion problem, sacrificing completeness for scalability. Most of this research looks heuristics for executing only a small subset of possible paths while minimising the impact this has on fault finding. Some typical solutions are:

1. *Coverage-based search*, which iteratively chooses the path with the highest number of unexplored statements/branches to achieve branch coverage rather than path coverage, or the paths with code that was just added to the program.
2. *Random path search*, which works surprisingly well, but not as systematically as coverage-based search.

Loops!: In our examples, the programs contained no loops. Loops further exacerbates the path explosion problem by giving us a potentially infinite number of paths. Further to this, while we can use testing heuristics, such as the 0-1-many rule to select a small set of paths from a loop, executing certain paths after a loop may require the loop to execute a specific number of times. The automatic search algorithms employed by symbolic execution tools are not intelligent enough to determine these, and proceed rather blindly, resulting in the tool getting "stuck".

Unsolvable paths: A final issue is with unsolvable paths: those paths that cannot be symbolically executed, effectively halting exploration of the path. Paths are typically unsolvable for two reasons:

1. *Unsolvable constraints*: a branch or instruction contains a statement that is outside of the scope of the constraint solver. For example, many state-of-the-art constraint solvers consider only linear constraints (e.g. of the form $z \geq 2x + 4y$) over integers, meaning that they can handle only integers and sometimes arrays [6]. So, when encountering a program with real numbers or nonlinear constraints, the symbolic execution will fail at a point that these are used, or must revert to approximate evaluation, which may be unsound.

Skip to main content

2. *External calls*: if a program contains a call to an external function, such as a library, or a function that reads user input, then it cannot reason about this symbolically, as it does not have access to the source code. As such, the symbolic execution algorithm must either halt, or approximate what the answer will be for the output.

The KLEE symbolic execution tool has its own model of the Linux system library, which it uses to get around this problem; however, this is not a generalisable solution.

10.3.8. Tools

There are a handful of mature symbolic execution tools, primarily built from research prototypes.

Java PathFinder[7] is a model checker and symbolic execution tool for Java, built and maintained at the NASA Ames Research Centre.

KLEE[8] is the most mature symbolic execution tool for LLVM (Low Level Virtual Machine), and many research projects have built on KLEE since its release in 2008. KLEE is still actively maintained.

Sypy[9] is a symbolic execution tool for Python, which has not been systematically evaluated. It appears that the development on the tool has halted.

Footnotes

[6]: https://stp.githul

[7]: http://babelfish.arc.n pf

[8]: http://klee.githul

[9]: diffoperator/S

10.4. Dynamic symbolic execution

Dynamic symbolic execution (DSE) is a test input generation technique that is a cross between random testing and symbolic execution, with the aim of overcoming the weaknesses of each of these.

It is perhaps more accurately described as a combination of dynamic execution and symbolic execution. Essentially, a DSE tool runs a concrete input on a program, and simultaneously performs symbolic execution on the path that this input executes. It then uses the path constraint generated by the symbolic execution to generate a new test.

10.4.1. Overview

DSE is used for dynamic test input generation. It uses symbolic execution, but the approach to test input generation is different to that of symbolic execution.

Example 72

We illustrate this with an example. Consider the following program, with a fault at the line that says "abort()":

```
1. void good_bad(char s[4]) {
2.   int count = 0;
3.   if (s[0] == 'b') count++;
4.   if (s[1] == 'a') count++;
5.   if (s[2] == 'd') count++;
6.   if (s[3] == '!') count++;
7.   if (count > 3) abort();  //fault
8. }
```

This program will only fail on the input "bad!". Using random testing, random arrays of four characters will be generated, but encountering the fault at line 7 is highly unlikely. If we consider a 32-bit architecture, the probability of randomly generating the string "bad!" requires us to randomly choose 'b' at index 0 (probability of 1 in 2^8, 'a' at location 1 (probability of 1 in 2^8), etc., leaving us with the probability of 1 in 2^{32} that the fault will be encountered.

If we use dynamic symbolic execution on that program, we first generate a random input, and symbolically execute the path on that input while also executing the concrete input. For illustration's sake, let's consider that the randomly generated input is the string "good". The path constraint will be: $s_0 \neq$ `b`, $s_1 \neq$ `a`, $s_2 \neq$ `d`, $s_3 \neq$ `!`, resulting in `count` being 0, and the fault not being executed.

Next, we use a constraint solver to generate a *new* test. With symbolic execution, we would use the path constraint to generate a concrete input for the path, however, we have already executed a concrete input for that path: the input "good". So instead, we generate a new test by *negating* one of the branches in the path constraint; e.g. the final branch: $s_0 \neq$ `b`, $s_1 \neq$ `a`, $s_2 \neq$ `d`, $s_3 =$ `!`. This will generate a new input, such as "goo!".

We can systematically repeat this for all 16 combinations of branches, symbolically executing each time, which will provide us with complete path coverage. One of the 16 combinations will be the constraint $s_0 =$ `b`, $s_1 =$ `a`, $s_2 =$ `d`, $s_3 =$ `!`, which generates the input "bad!", and will execute the fault.

Skip to main content

Example 73

The above example illustrates how DSE differs from symbolic execution, but does not illustrate why using concrete values can be advantageous. For this, consider the following example, which contains a call to an external function called `hash(char s[8])`, which calculates a hash value for a string:

```
1.  void hash_example(int x, char s[8])
2.  {
3.    if (x == hash(s)) {
4.      //do something good
5.    }
6.  }
```

In this example, the input `x` must be equal to the hash value of `s` to execute the code block inside. Such code is common in applications with security restrictions, and to test most of the application's functionality, the branch needs to evaluate to true for most tests.

Now, consider a case in which we cannot symbolically execute `hash`; either because we do not have the source code (it is an external function), or its logic is too complex for the constraint solver being used. If we used random testing, we are unlikely to execute a test such that the branch is true. If we used symbolic execution, the execution would halt at the branch.

However, if we use DSE, we can overcome this problem. First, the DSE algorithm will pick random values for `x` and `s`; for example, `x = 1019` and `s = "s3fpmi"`. When the branch is symbolically evaluated, we cannot evaluate `hash(y)`, so we instead record the concrete value; e.g. 74233. Next, we add the concrete value to the path constraint instead of the symbolic value, to get the condition $x \neq 74223$.

To generate a new test, we negate this path constraint ($x = 74223$), solve this constraint, and get the input `x = 74223`, and the branch will evaluate to true, executing the code block instead the if statement. Brilliant!

Of course, if *both* sides of the equality are not symbolically executable (e.g. `hash(x) == hash(y)`), this will not work; but this solution works for a large number of cases in which we can access the concrete values but not the symbolic values of function calls.

In summary, the difference between symbolic execution and DSE is two things:

1. The concrete input is executed simultaneously with the symbolic input, instead of generating the test after symbolically executing a path. The first input is chosen arbitrarily (perhaps randomly).
2. Path constraints are modified to generate a *new* test after each path is executed.

10.4.2. Advantages

10.4.2.1. Advantages over random testing

Clearly, the advantage of DSE over random testing is that it is not so random. While the first test case is random, using the program's control-flow to explore the input space provides a much better coverage of the program structure than with random testing.

10.4.2.2. Advantages over symbolic execution

Using DSE to generate test inputs has a major advantage over using just symbolic execution: when an unsolvable path (e.g. unsolvable constraint or external function call) is encountered, DSE uses the concrete values for those variables that cannot be solved, and adds these to the symbolic state. This effectively under-approximates the symbolic values of those variables, but it allows execution to continue.

A second advantage is that it opens up to new search strategies to deal with the path explosion problem. For example, for their SAGE DSE tool[10], Microsoft researchers proposed a technique known as *generational* search. In generational search, one symbolically executed path is used to generate multiple new tests.

For example, in our good/bad example above, instead of generating one new test from the path constraint $s_0 \neq$ `b`, $s_1 \neq$ `a`, $s_2 \neq$ `d`, $s_3 \neq$ `!`, and executing this using DSE, SAGE generates four new inputs by negating *each* of the constraints in the path constraint, giving four new path constraints:

1. $s_0 =$ `b`, $s_1 \neq$ `a`, $s_2 \neq$ `d`, $s_3 \neq$ `!` $\Rightarrow$ "bood"

Footnotes

[10]: See https://patricegodefr ublic_psfiles/ndss20

Skip to main content

3. $s_0 \neq$ `b`, $s_1 \neq$ `a`, $s_2 =$ `d`, $s_3 \neq$ `!` $\Rightarrow$ "godd"
4. $s_0 \neq$ `b`, $s_1 \neq$ `a`, $s_2 \neq$ `d`, $s_3 =$ `!` $\Rightarrow$ "goo!"

This gives four new inputs having executed only one symbolic path, reducing the cost of DSE significantly. As these four tests are executed, the next "generation" is born, which negates the constraints in the path constraints that have not been executed previously. For example, the test "bood" will generate three more constraint that result in the test inputs: (1) "baod"; (2) "bodd"; and (3) "boo!". The test "good" is not generated again because that path has already been covered. Another generation is executed, until we get the test "bad!".

This generational search algorithm could allow DSE to scale much better than using a more complete algorithm, such as depth-first or breadth-first search.

10.4.3. Limitations

DSE still suffers from all of the same limitations as symbolic execution: path explosion, expensive constraint solving, loops, and unsolvable paths. However, the use of concrete information during execution helps to mitigate all of these at some level, significantly improving the scalability.

However, DSE suffers from a problem that is not found in symbolic execution: *divergence*. Because a concrete input is used to drive which path is executed next, a DSE algorithm does not fork at each branch, but instead just collects the symbolic values of the path being executed. However, if a new test is generated and the path it executes is different to the path we expected, we say that a *divergence* has occurred. This is caused by the limitations in the constraint solvers used.

For example, consider the following program:

```
1.  void divergent(int x, int y)
2.  {
3.    int s[4];
4.    s[0] = x;
5.    s[1] = 0;
6.    s[2] = 1;
6.    s[3] = 2;
7.    if (s[x] = s[y] + 2) {
8.      abort(); //error
9.    }
10. }
```

If our first randomly generated input is `x = 0` and `y = 1`, then the branch statement at line 7 will be false: `s[x] = 0` and `s[y] + 2 = 2`. Because constraint solvers are not sophisticated enough to reason about symbolic addresses (or arrays) completely, the concrete values of `s[0]` (0 is the concrete value of `x`) and `s[1]` (1 is the concrete value of `y`) are used instead. The symbolic value of `s[0]` is `X`, and for `s[1]` it is 0. Because `x == 0 + 2` is false, the path constraint generated is $x \neq 2$. Negating this results in the new test inputs `x = 2, y = 1`. However, because the value of `s[x]` is dependent on the input, we get the values `s[x] = 1` and `s[y] + 2 = 2`, so the branch at line 6 again evaluates to false, and the fault is missed. The DSE algorithm will then terminate, and will miss the case `x = 3, y = 1`, which finds the fault.

In this case, concretisation results in divergence, which would not happen using symbolic execution. However, considering that the constraint solver is not able to reason about symbolic addresses, the symbolic execution could not continue at this point, so using concretisation will help in some cases.

Recent research studies how to analyse such problems with arrays and pointer arithmetic to provide better solutions, but these do not always work due to the complicated nature of pointers.

10.4.4. Tools

The most successful DSE tool to date is Microsoft SAGE [11]. SAGE is not available for download, however, it has been used successfully internally at Microsoft since 2008, finding over one third of all security vulnerabilities found in Windows 7. Microsoft currently host a lab of over 200 machines dedicated to running SAGE over their development code.

S^2E[12] is an open-source DSE tool for the LLVM, built on KLEE, which contains a sophisticated approach for generating sequences of method calls to find vulnerabilities in programs.

CREST[13] and jCUTE[14] are open-source DSE tools for C and Java respectively, primarily designed to allow researchers to modify and experiment with DSE.

Footnotes

[11]: http://research.microsoft.com/en-us/um/people/pg/publications/ndss2008-in-one-slide.pdf

[12] http://s2e.epfl.ch

[13] http://jburnim.github.com/crest

[14] http://osl.cs.illinois.edu/jcute/

Skip to main content

10.5. Into the future...

Symbolic analysis, and dynamic symbolic execution in particular, are likely to play a significant role in testing in the future, as the complexity of software systems increase to the point where manual test generation is simply not feasible to achieve high coverage.

As constraint solvers continue to improve, the effect is passed on to symbolic execution tools who build on these. Constraint solvers are improving at a remarkable rate as researchers find new tricks to improve reasoning without the loss of precision, and find heuristics to give approximate answers. Further, researchers are increasing the domains in which constraint solvers can find answers in reasonable times, including real numbers and certain nonlinear domains.

Symbolic execution and DSE are exciting research topics, with research being done in this department on those topics, as well as other symbolic reasoning approaches such as abstract interpretation.